Criando testes unitários com JUnit

Aluno: João Vitor Pavanelli
Curso: 3º D.S. (Desenvolvimento de Sistemas), período noturno
Instituição: Etec Zona Leste
Disciplina: Desenvolvimento de Sistemas II

Questões para responder

1. Qual é a função da anotação @Test?

A anotação @Test serve para avisar o JUnit que aquele método é um teste. Com ela, o JUnit encontra o método sozinho e executa quando mandamos rodar os testes, sem precisar de um main para chamar cada um.

2. Para que serve o método assertEquals()?

O assertEquals() compara o valor que a gente espera com o valor que o método realmente devolveu. Se os dois forem iguais, o teste passa, e se forem diferentes, o teste falha. Nos testes com números decimais (double), o terceiro parâmetro (0.001) é a margem de erro aceita na comparação.

3. Por que os testes falharam após a alteração da operação matemática?

Porque trocamos a multiplicação pela soma, e aí o método passou a devolver outro resultado. Por exemplo, no teste de 3 produtos de R$ 15,00 o esperado era 45, mas com a soma o método devolveu 18 (15 + 3). Como o resultado ficou diferente do esperado, o assertEquals() acusou o erro e os testes falharam.

4. Qual é a vantagem de executar testes automatizados após modificar um método?

A vantagem é descobrir na hora se a alteração quebrou alguma coisa, sem precisar conferir tudo na mão. Isso economiza tempo, evita que um erro vá parar no sistema final e dá mais segurança para mexer no código, porque se algo der errado os testes avisam.

Código da classe CalculadoraCompra

```
public class CalculadoraCompra {

    public double calcularTotal(double valor, int quantidade) {
        return valor * quantidade;
    }
}
```

Código da classe CalculadoraCompraTest

```
import org.junit.Test;
import static org.junit.Assert.*;

public class CalculadoraCompraTest {

    @Test
    public void testarCompraDeUmProduto() {
        CalculadoraCompra calc = new CalculadoraCompra();
        double resultado = calc.calcularTotal(10.0, 1);
        assertEquals(10.0, resultado, 0.001);
    }

    @Test
    public void testarCompraDeVariosProdutos() {
        CalculadoraCompra calc = new CalculadoraCompra();
        double resultado = calc.calcularTotal(15.0, 3);
        assertEquals(45.0, resultado, 0.001);
    }

    @Test
    public void testarCompraDeCincoProdutos() {
        CalculadoraCompra calc = new CalculadoraCompra();
        double resultado = calc.calcularTotal(8.0, 5);
        assertEquals(40.0, resultado, 0.001);
    }

    @Test
    public void testarCompraDeDezProdutos() {
        CalculadoraCompra calc = new CalculadoraCompra();
        double resultado = calc.calcularTotal(2.5, 10);
        assertEquals(25.0, resultado, 0.001);
    }
}
```